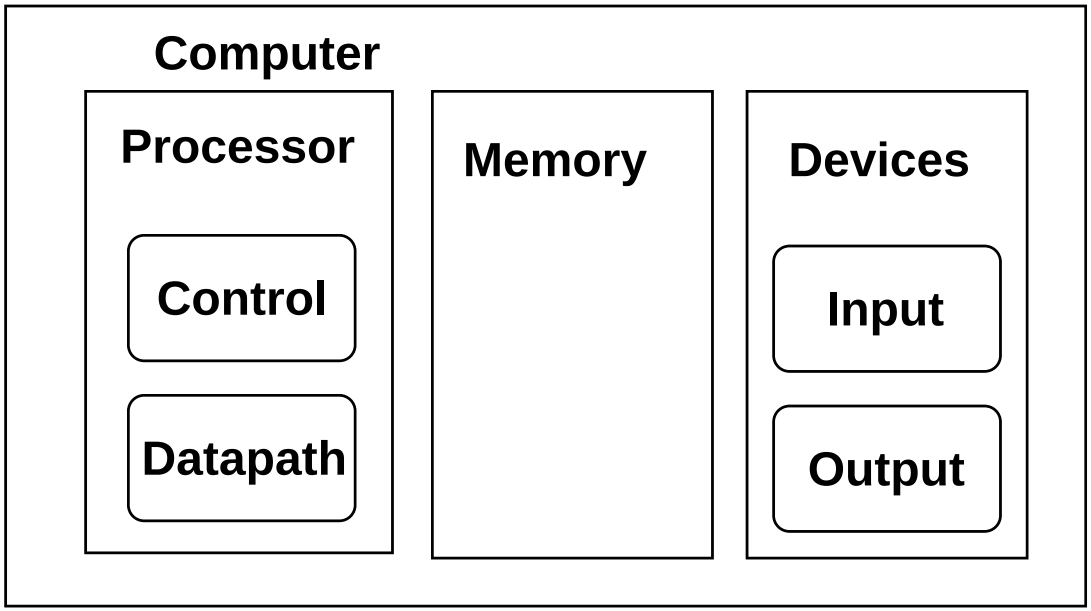
Datapath & Control

Readings: 4.1-4.4



Datapath: System for performing operations on data, plus memory access.

Control: Control the datapath in response to instructions.

Simple CPU

Develop complete CPU for subset of instruction set

Memory: `LDUR, STUR`

31 30 29 28 27 26 25 24 23 22 21	20 19 18 17 16 15 14 13 12	11 10	09 08 07 06 05	04 03 02 01 00
Opcode	DAddr9	00	Rn	Rd

Branch: `B`

31 30 29 28 27 26	25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10 09 08 07 06 05 04 03 02 01 00
Opcode	BrAddr26

Conditional Branch: `CBZ`

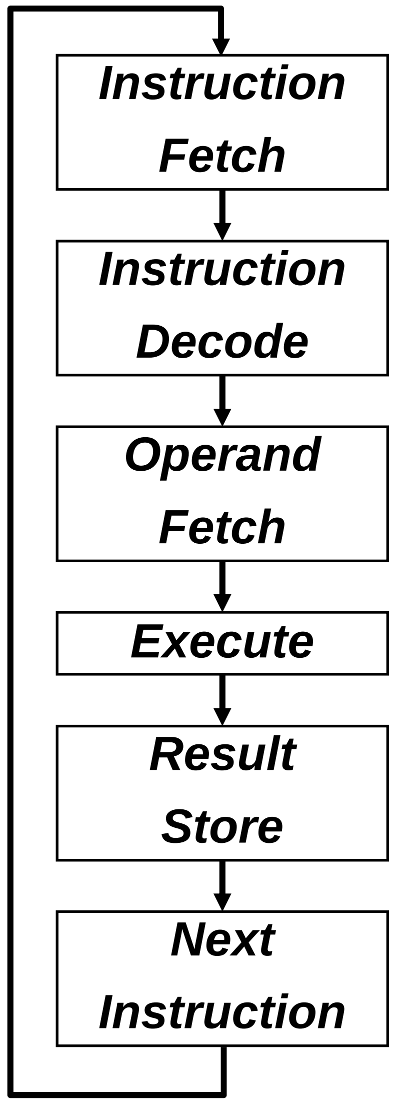
31 30 29 28 27 26 25 24	23 22 21 20 19 18 17 16 15 14 13 12 11 10 09 08 07 06 05	04 03 02 01 00
Opcode	CondAddr19	Rd

Arithmetic: `ADD, SUB`

31 30 29 28 27 26 25 24 23 22 21	20 19 18 17 16	15 14 13 12 11 10	09 08 07 06 05	04 03 02 01 00
Opcode	Rm	SHAMT	Rn	Rd

Most other instructions similar

Execution Cycle



Obtain instruction from program storage

Determine required actions and instruction size

Locate and obtain operand data

Compute result value or status

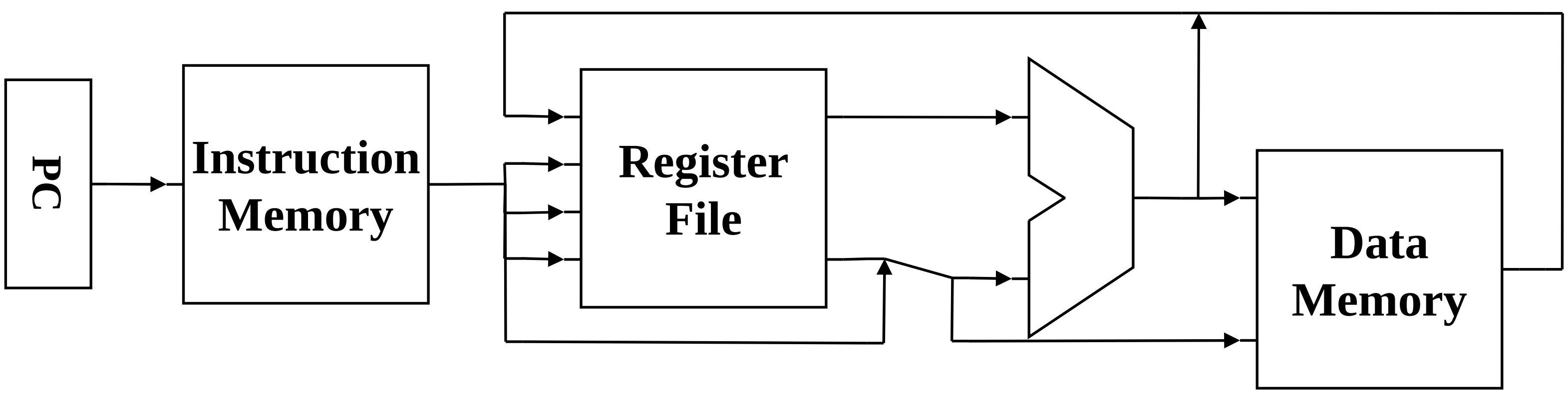
Deposit results in storage for later use

Determine successor instruction

Processor Overview

Overall Dataflow

- PC fetches instructions
- Instructions select operand registers, ALU immediate values
- ALU computes values
- Load/Store addresses computed in ALU
- Result goes to register file or Data memory



RTL & Processor Design

Convert instructions to Register Transfer Level (RTL) specification

```
RegA = RegB + RegC;
```

RTL specifies required interconnection of units, control

Math unit example:

(add): A = A + B; I++;

(mult): A = A * B: I++;

(hold): A = A; I++;

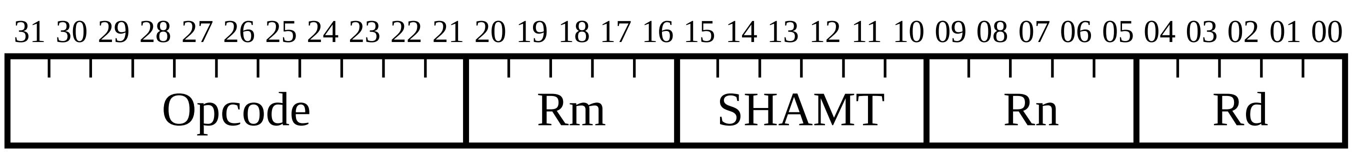
(init): A = Din; I++;

Instruction Fetch

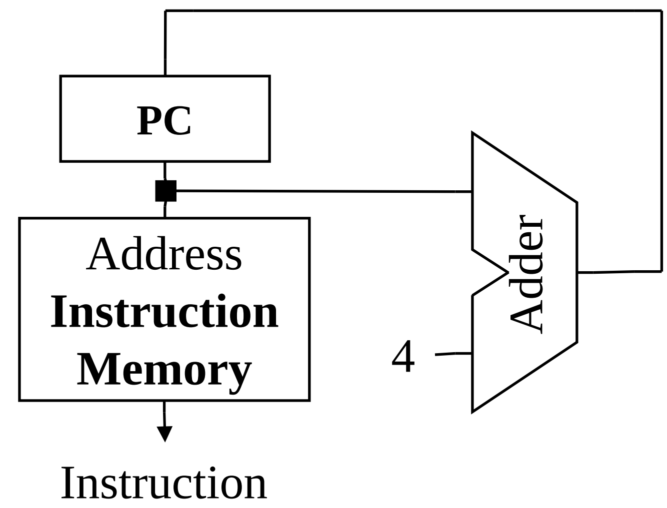
Add/Subtract RTL

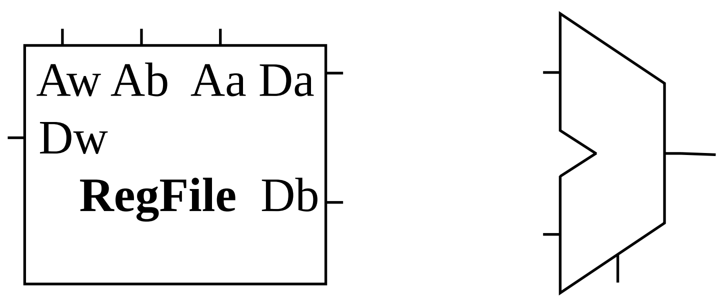
Add instruction: `ADD Rd, Rn, Rm`

Subtract instruction: `SUB Rd, Rn, Rm`



Add/Subtract Datapath



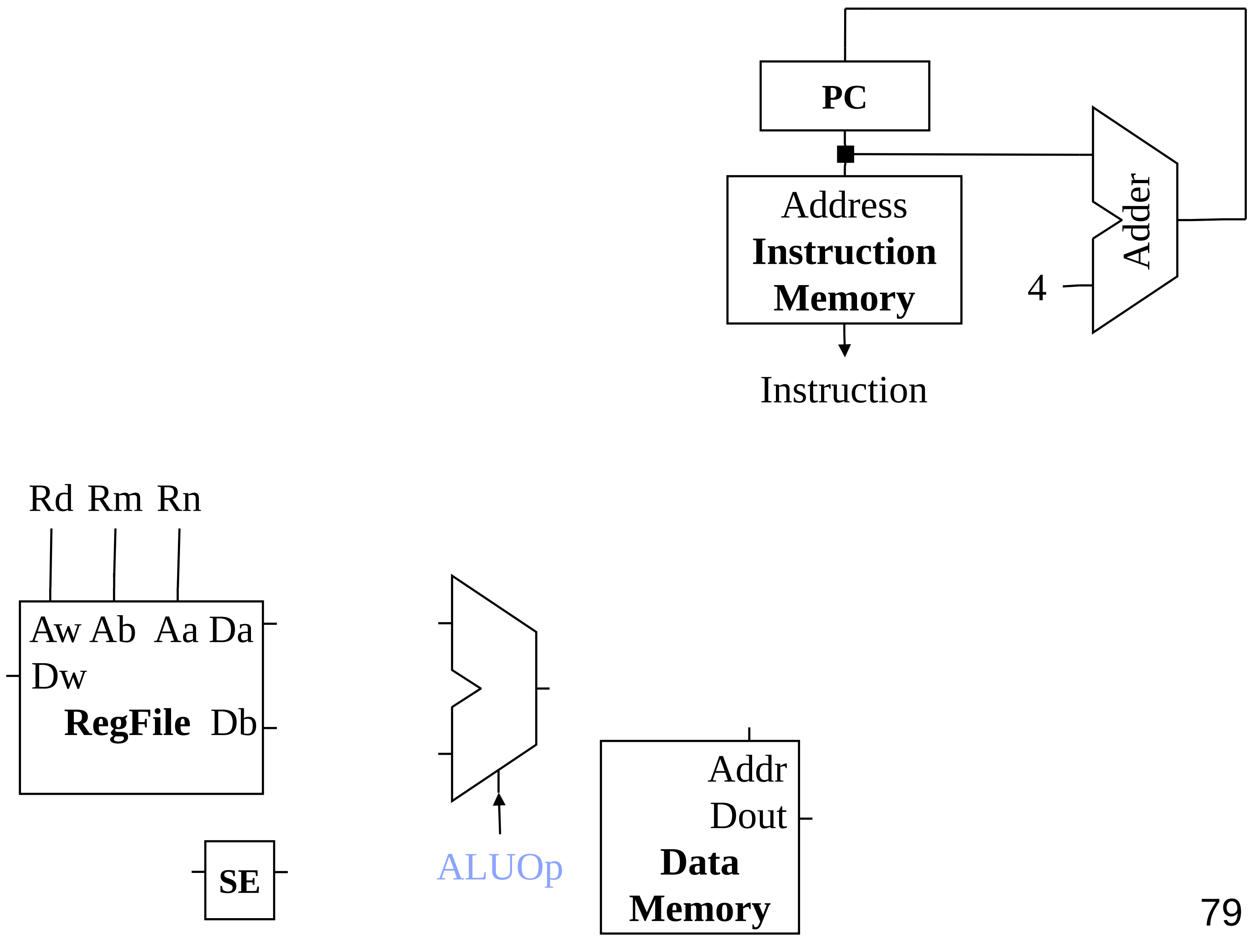


Load RTL

Load Instruction: `LDUR Rd, [Rn, DAddr9]`

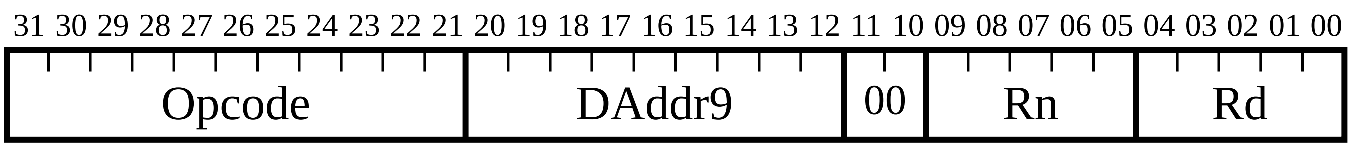
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
Opcode											DAddr9									00		Rn					Rd				

Datapath + Load

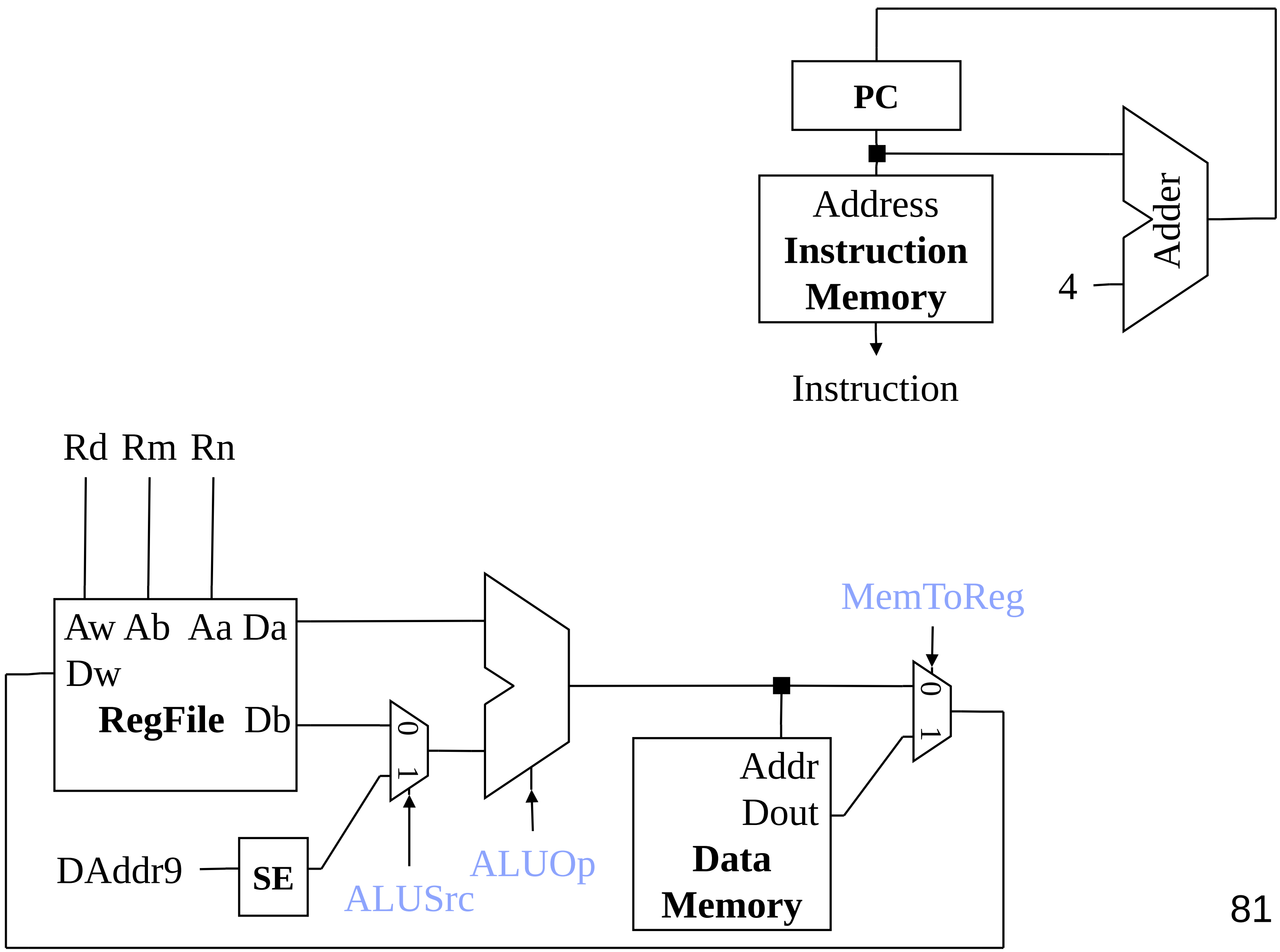


Store RTL

Store Instruction: `STUR Rd, [Rn, DAddr9]`



Datapath + Store

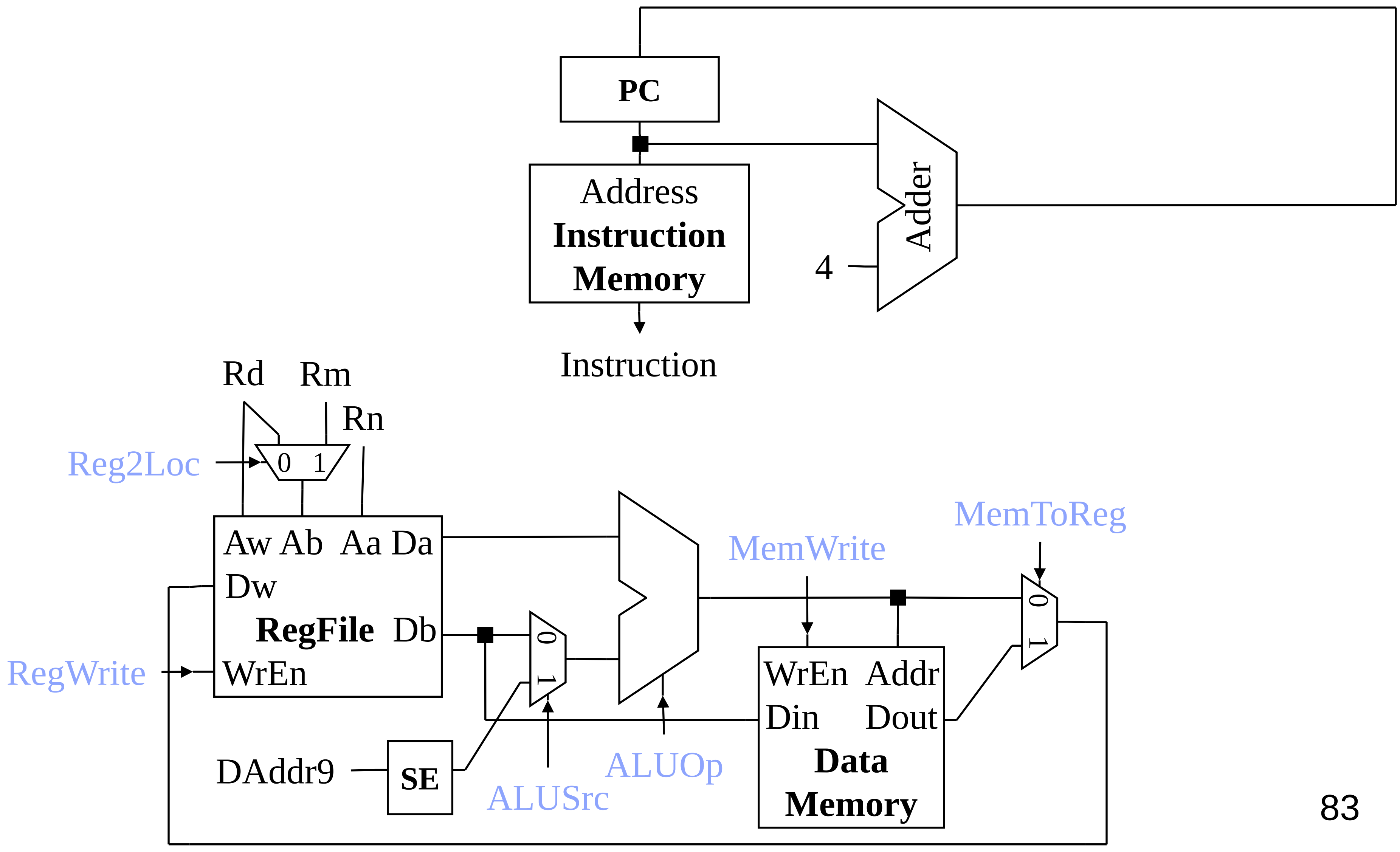


Branch RTL

Branch Instruction: `B BrAddr26`

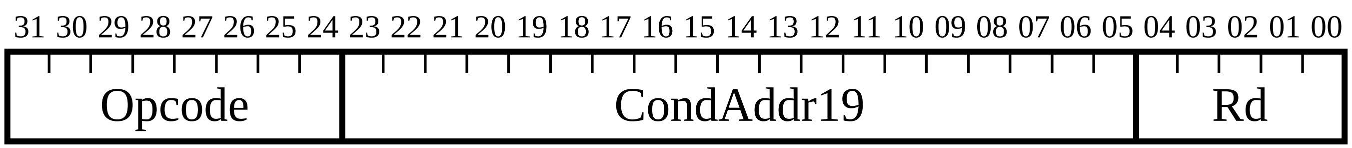
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	1	0	1	BrAddr26																									

Datapath + Branch

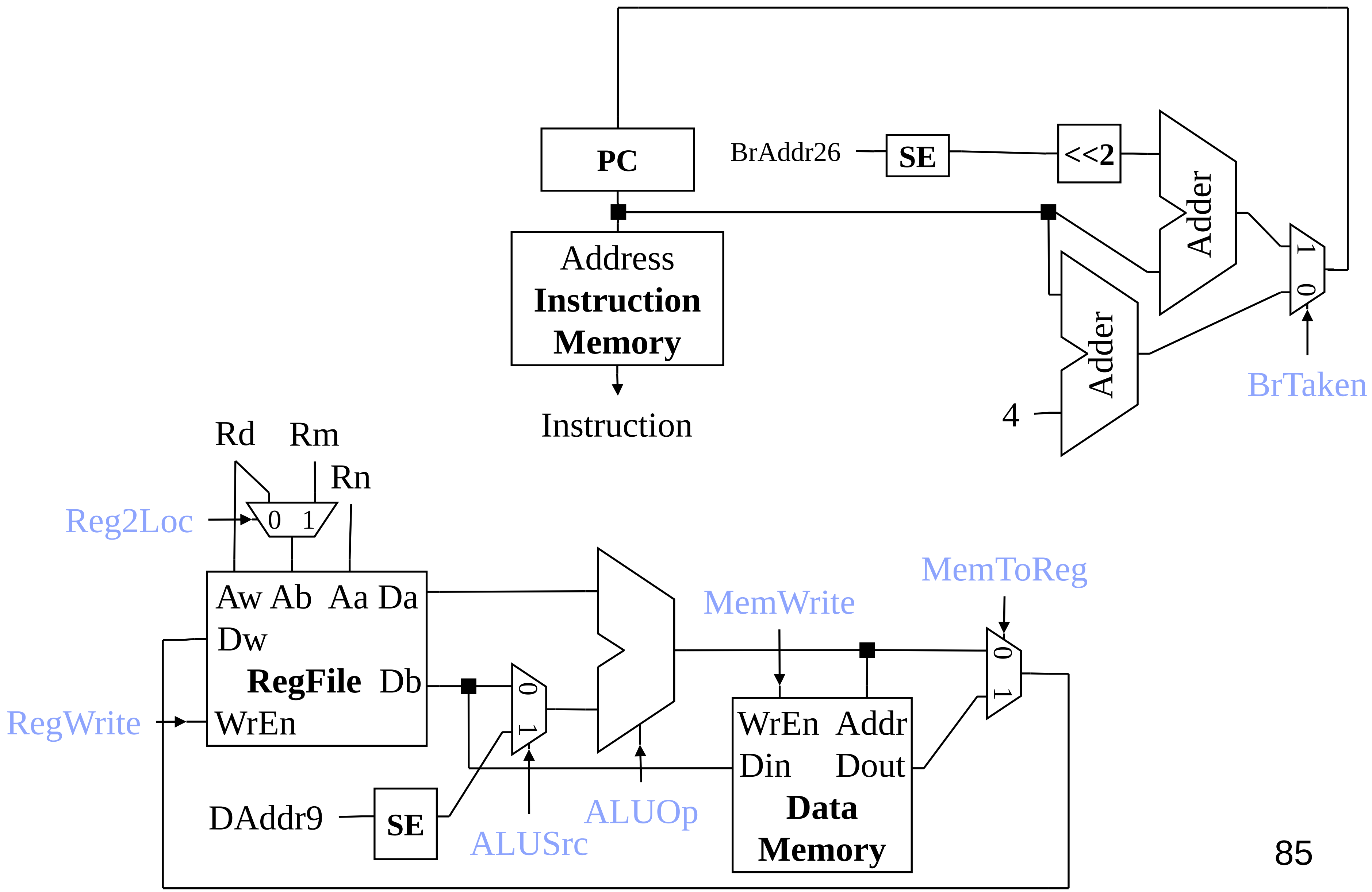


Conditional Branch RTL

Conditional Branch Instruction: `CBZ Rd, CondAddr19`



Datapath + Conditional Branch



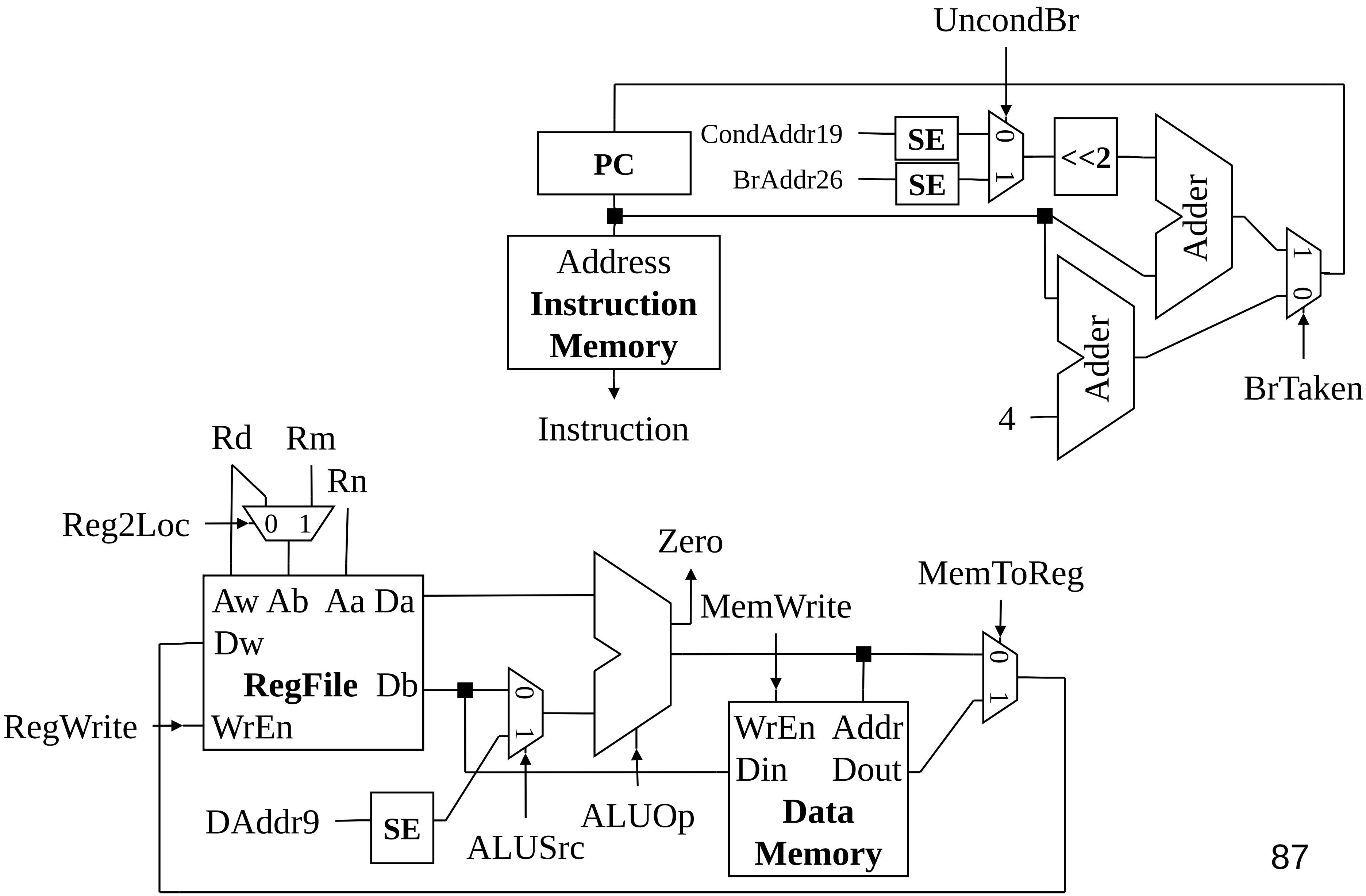
Control

Identify control points for pieces of datapath

- Instruction Fetch Unit
- ALU
- Memories
- Datapath muxes
- Etc.

Use RTL for determine per-instruction control assignments

Complete Datapath

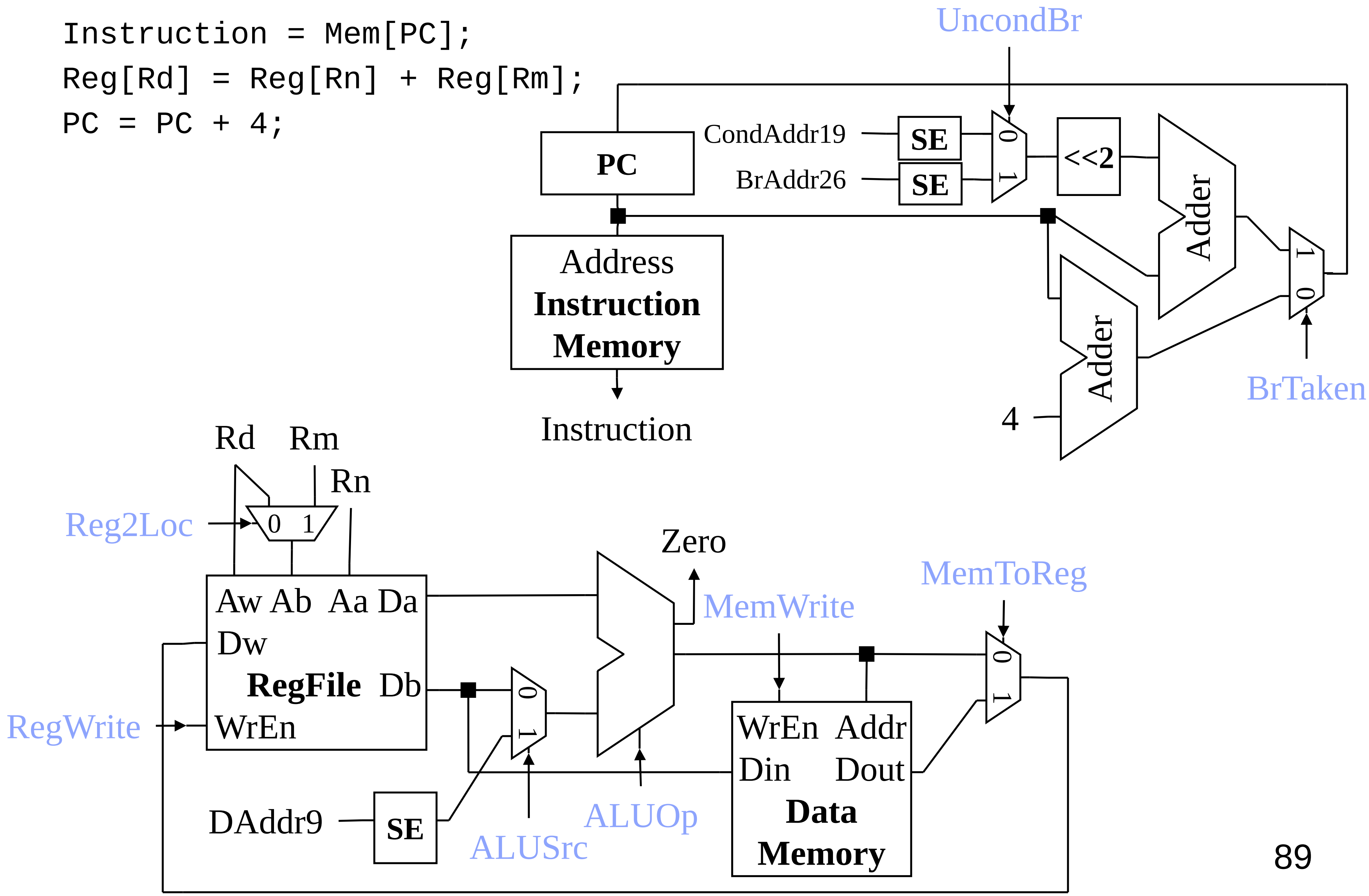


Control Signals

Opcode[31:26] Opcode[25:21]	100010 11000	110010 11000	111110 00010	111110 00000	000101 xxxxx	101101 00xxx
	ADD	SUB	LDUR	STUR	B	CBZ

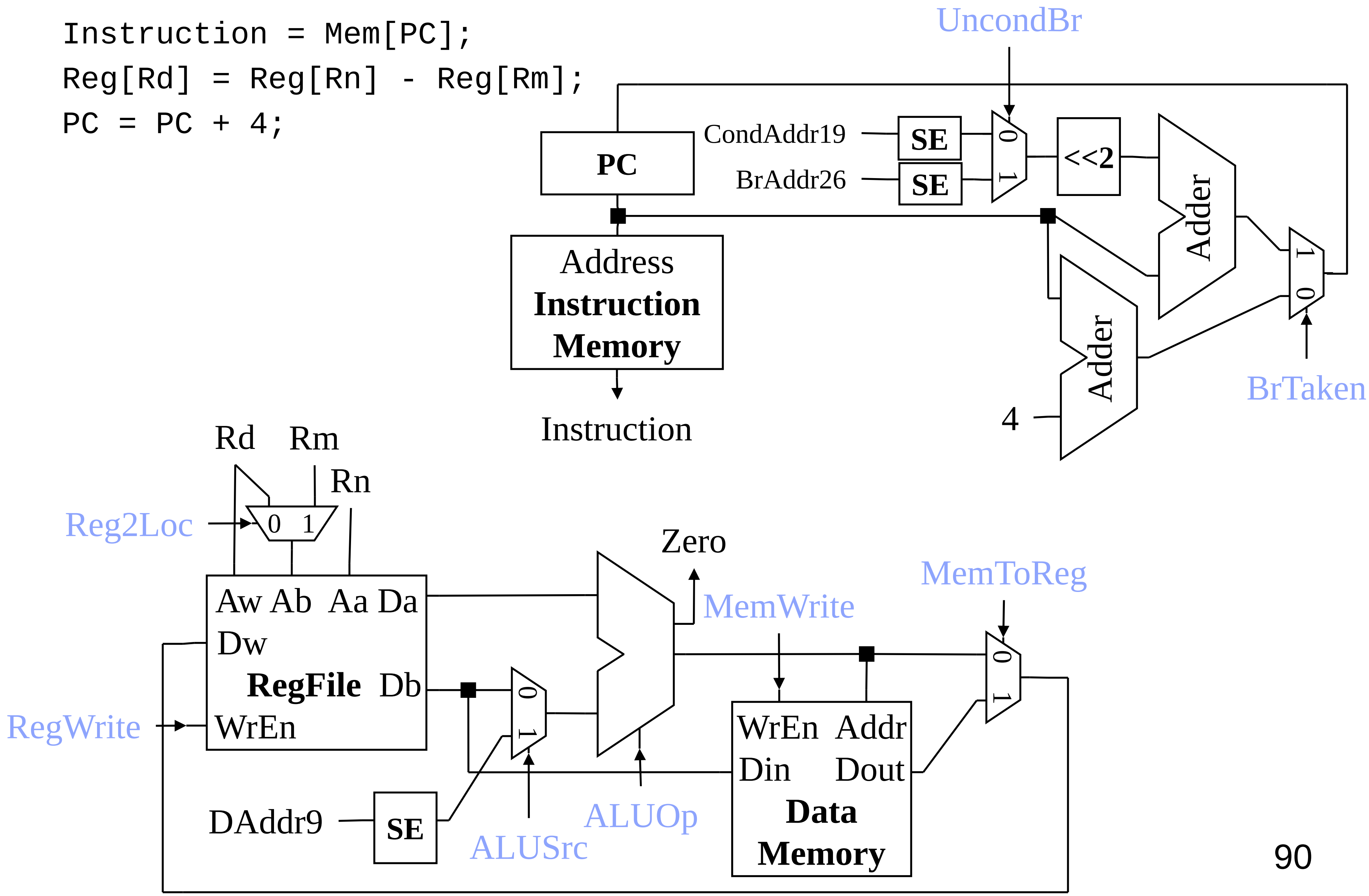
ADD Control

```
Instruction = Mem[PC];
Reg[Rd] = Reg[Rn] + Reg[Rm];
PC = PC + 4;
```



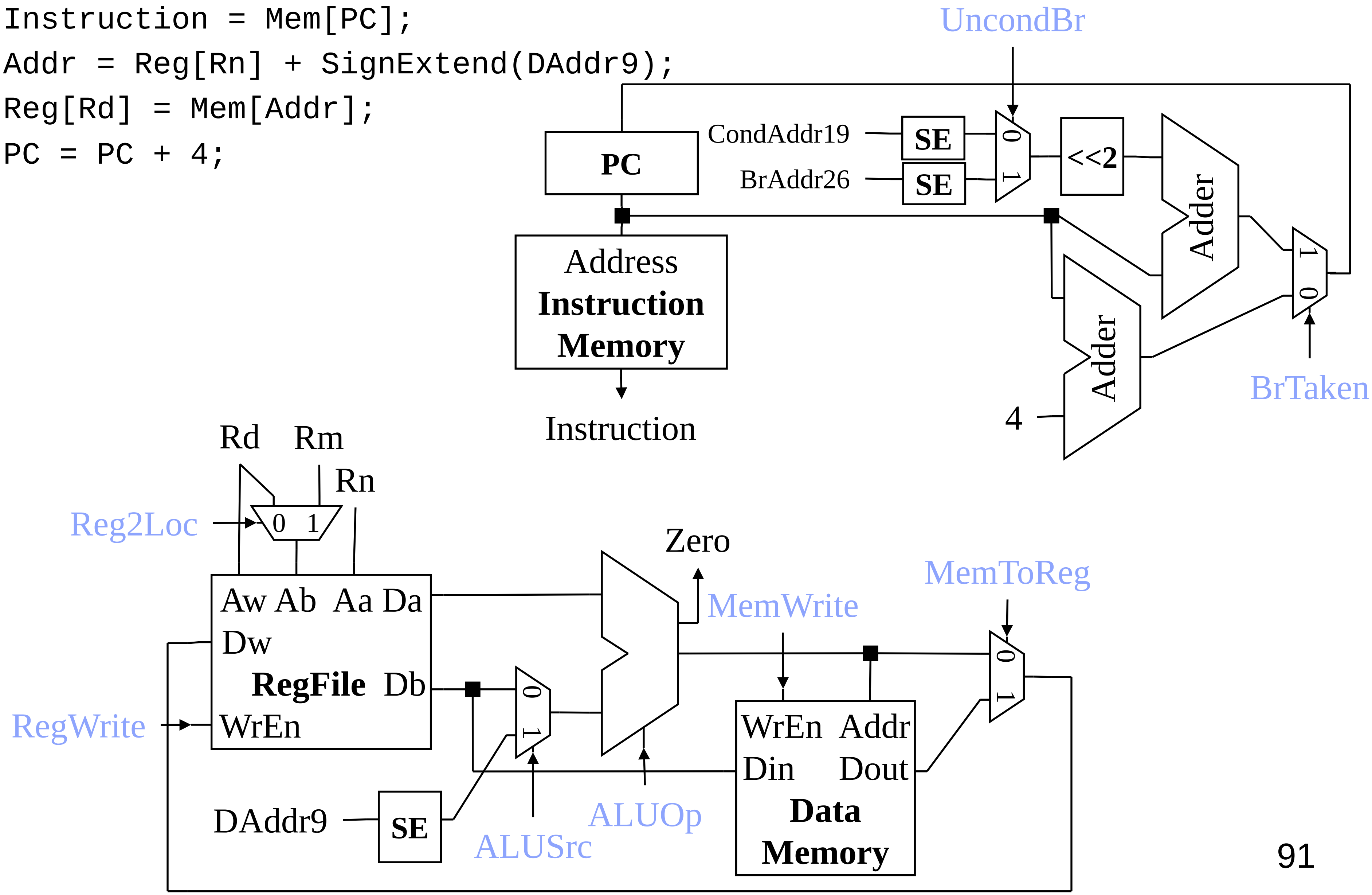
SUB Control

```
Instruction = Mem[PC];
Reg[Rd] = Reg[Rn] - Reg[Rm];
PC = PC + 4;
```



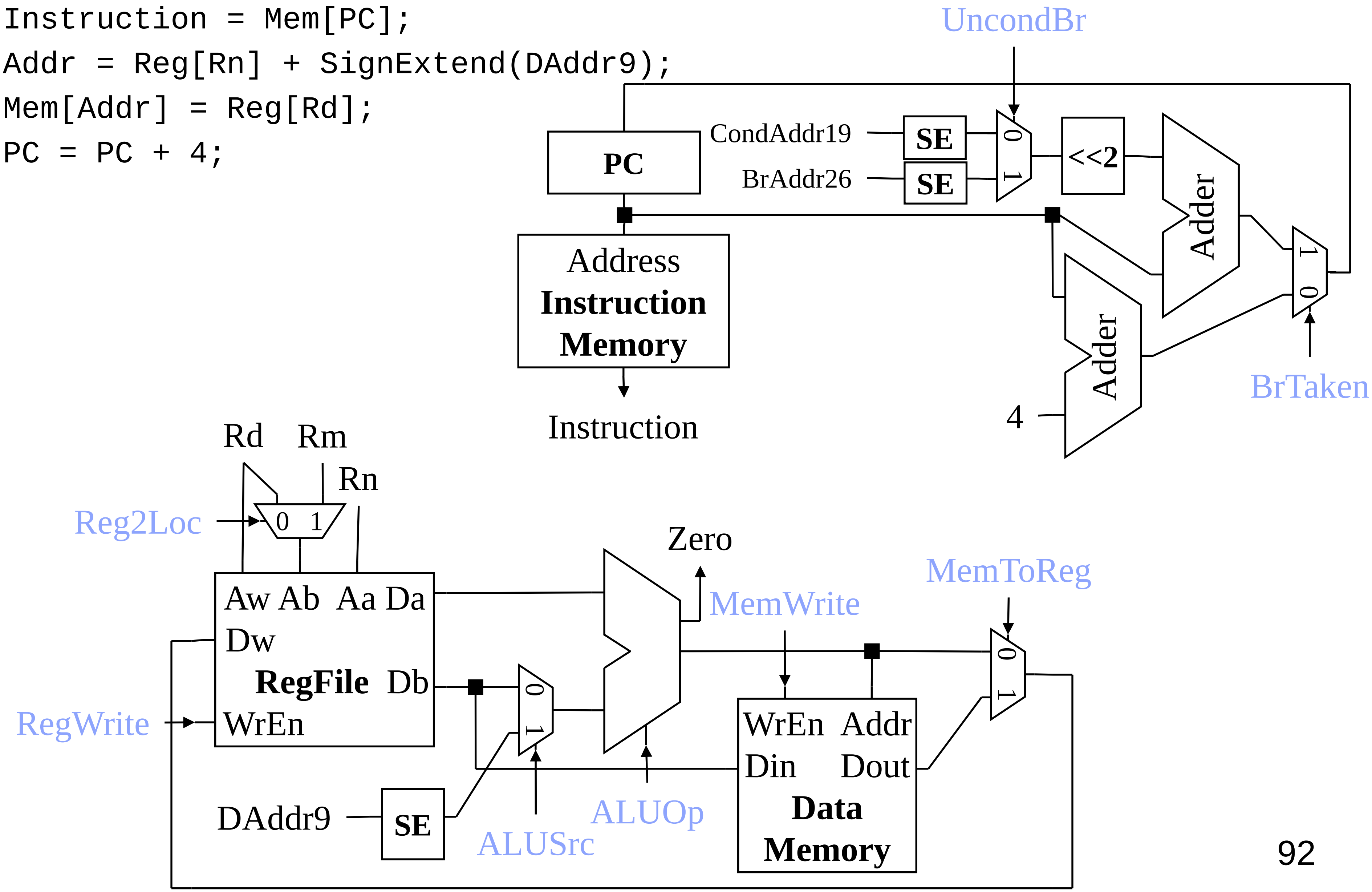
LDUR Control

```
Instruction = Mem[PC];
Addr = Reg[Rn] + SignExtend(DAddr9);
Reg[Rd] = Mem[Addr];
PC = PC + 4;
```



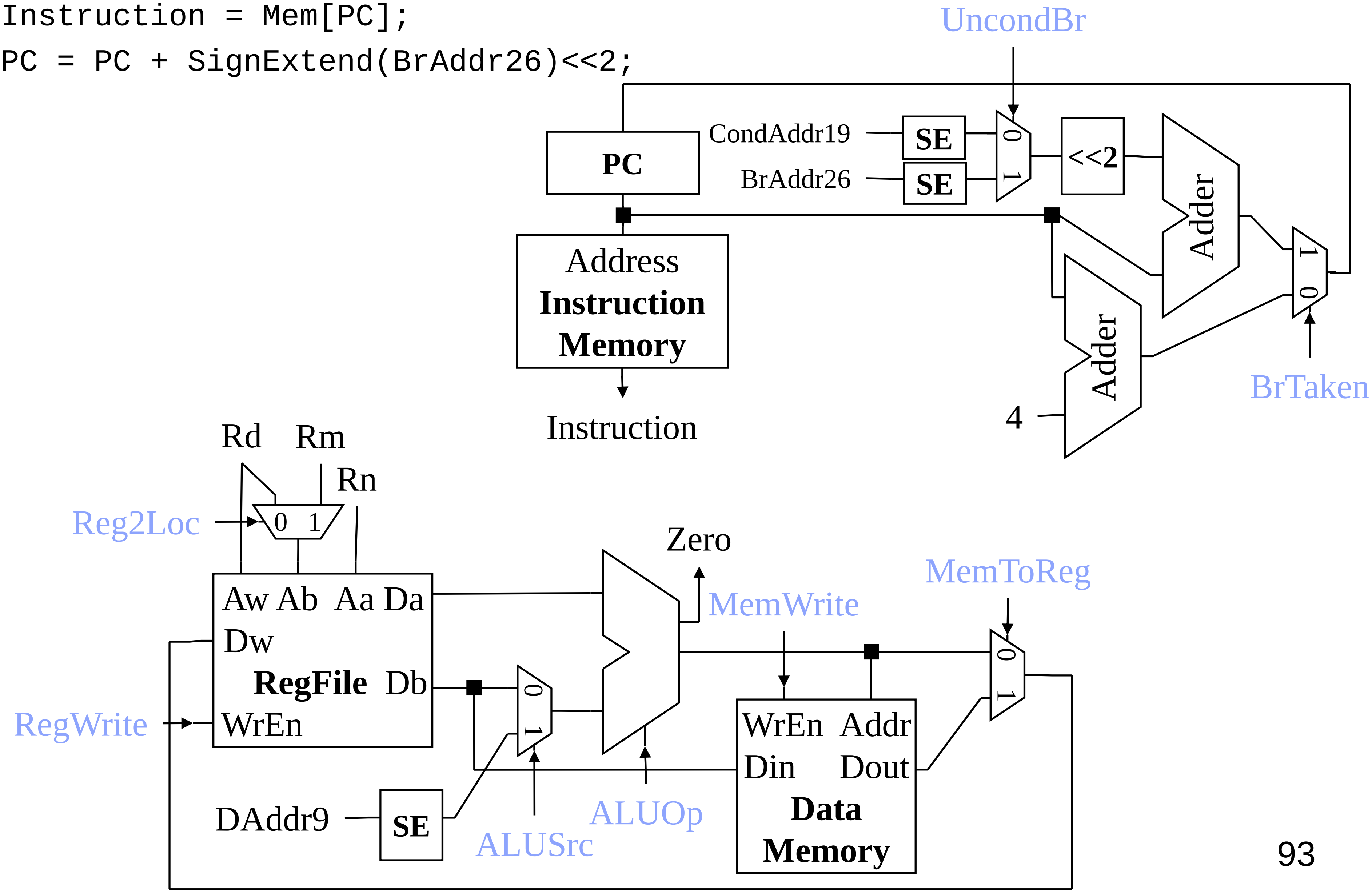
STUR Control

```
Instruction = Mem[PC];
Addr = Reg[Rn] + SignExtend(DAddr9);
Mem[Addr] = Reg[Rd];
PC = PC + 4;
```



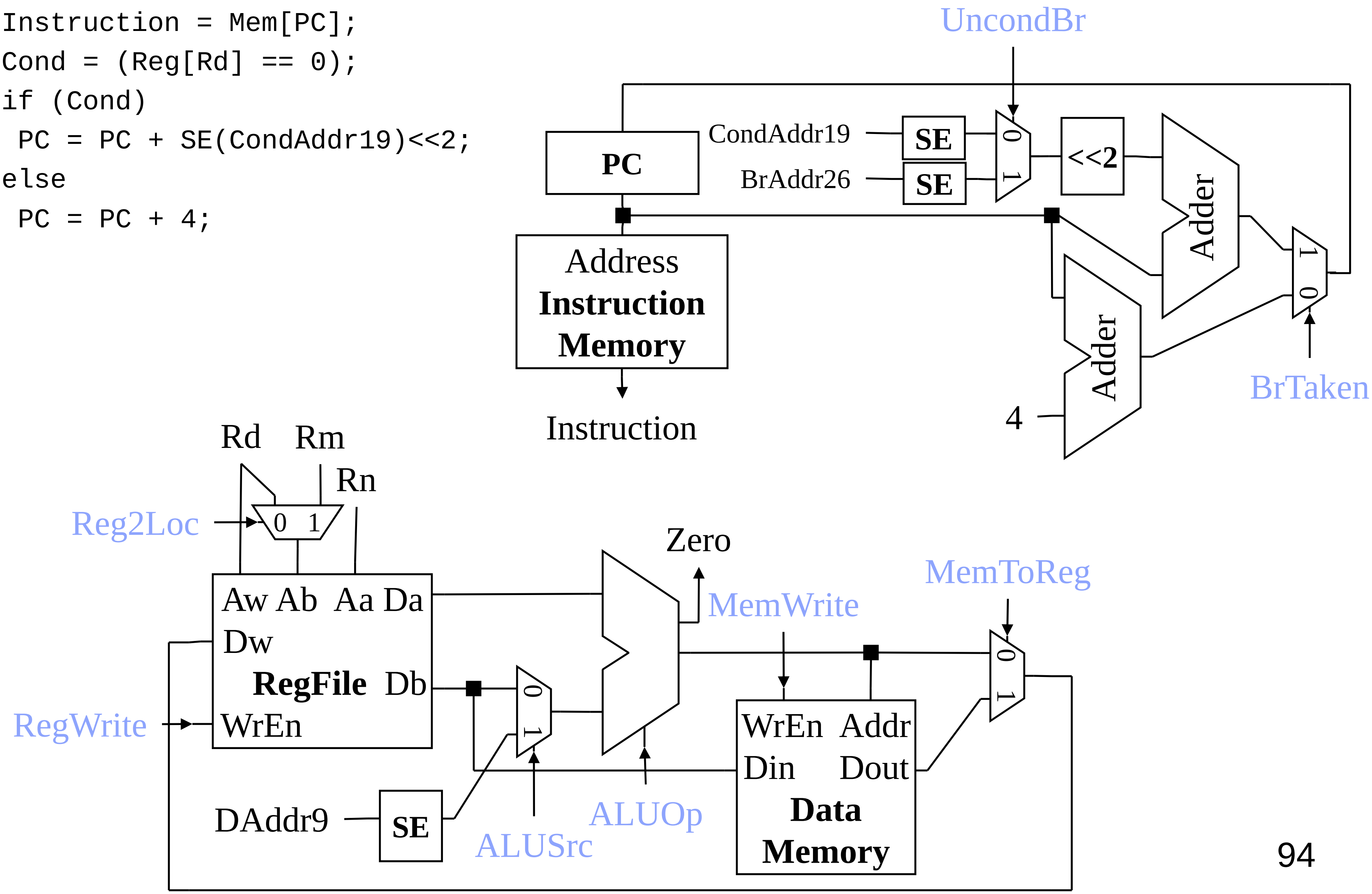
B Control

```
Instruction = Mem[PC];
PC = PC + SignExtend(BrAddr26)<<2;
```



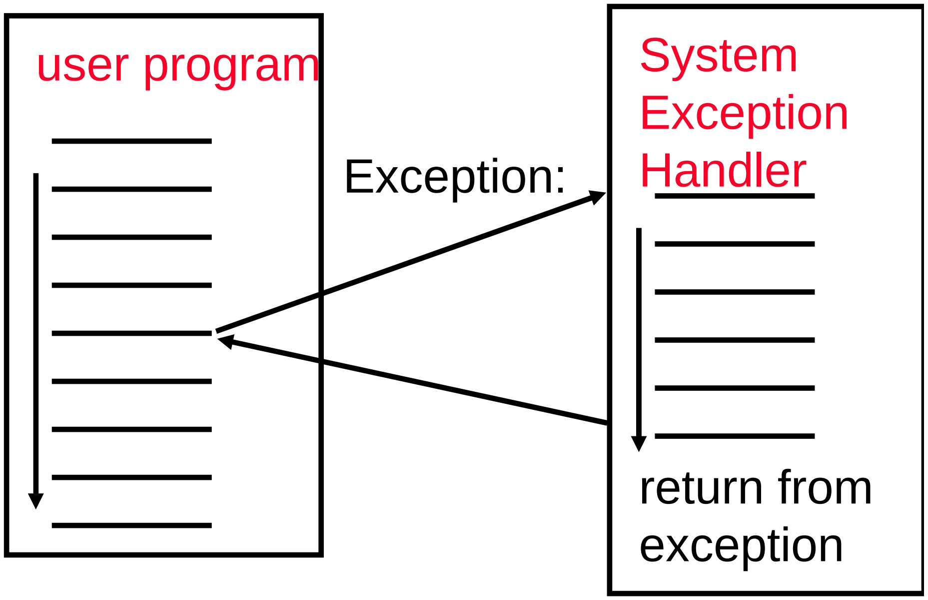
CBZ Control

```
Instruction = Mem[PC];
Cond = (Reg[Rd] == 0);
if (Cond)
 PC = PC + SE(CondAddr19)<<2;
else
 PC = PC + 4;
```



Advanced: Exceptions

Exception = unusual event in processor
- Arithmetic overflow, divide by zero, …
- Call an undefined instruction
- Hardware failure
- I/O device request (called an "interrupt")



Approaches
- Make software test for exceptional events when they may occur ("polling")
- Have hardware detect these events & react:
  - Save state (Exception Program Counter, protect the GPRs, note cause)
  - Call Operating System
    - If (undef_instr) PC = C0000000
    - If (overflow) PC = C0000020
    - If (I/O) PC = C0000040
    - …

Performance of Single-Cycle Machine

CPI?

ADD, SUB

PC	Instr. Memory	mux	Reg Read	mux	ALU	mux	Reg Setup

LDUR

PC	Instr. Memory	Reg Read	ALU	Data Memory	mux	Reg Setup

STUR

PC	Instr. Memory	Reg Read	ALU	Data Memory

CBZ

PC	Instr. Memory	mux	Reg Read	mux	ALU	mux	PC setup

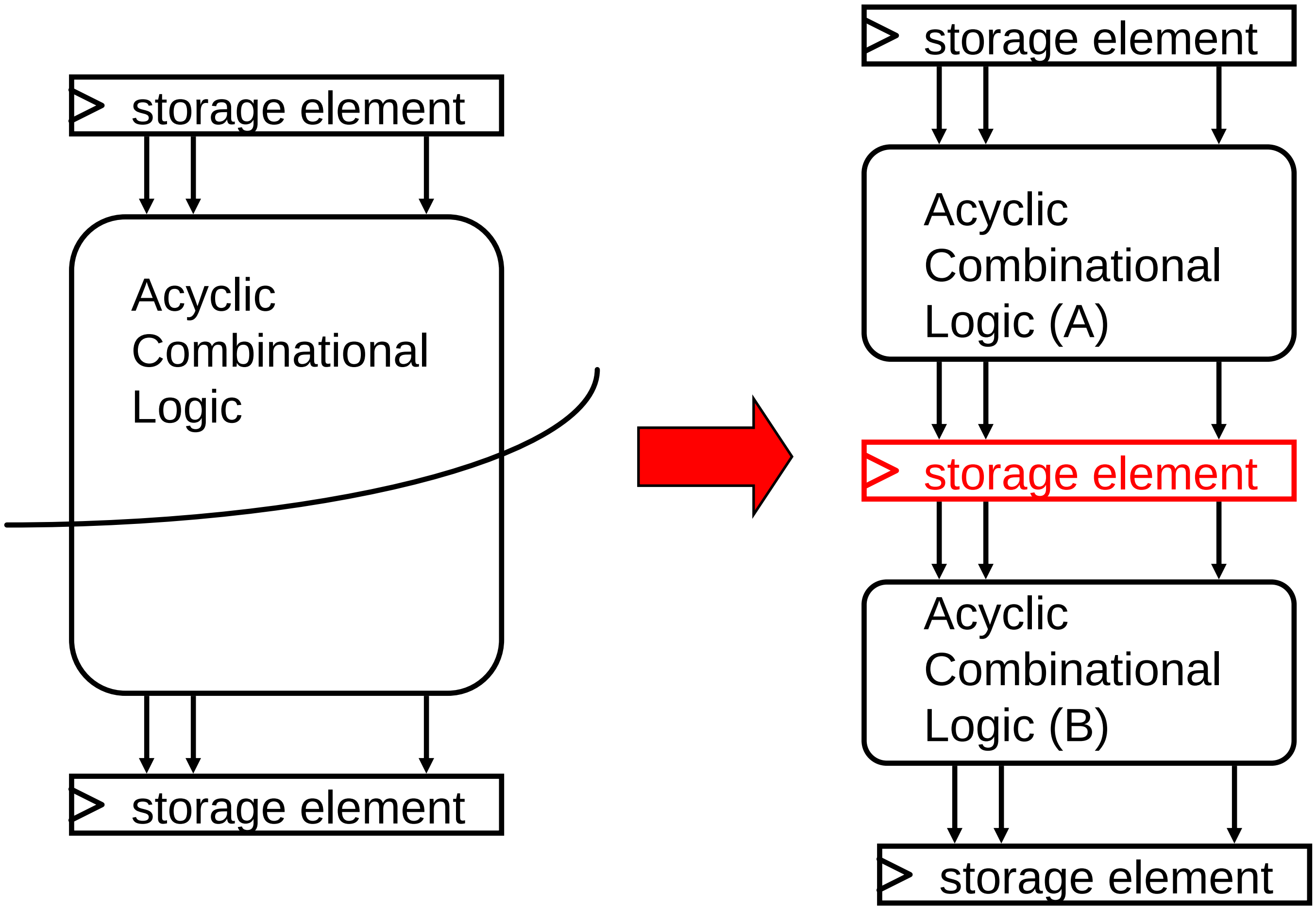
B

PC	Instr. Memory	Adder	mux	PC setup

Reducing Cycle Time

Cut combinational dependency graph and insert register / latch

Do same work in two fast cycles, rather than one slow one



Pipelined Processor Overview

Divide datapath into multiple stages